

# 從 App Engine Blobstore 遷移至 Cloud Storage (模組 16)

來源：<https://codelabs.developers.google.com/codelabs/cloud-gae-python-migrate-16-cloudstorage?hl=zh-tw>

步驟數：7

瞭解如何將 Python 2 App Engine ndb 應用程式的 Blob 用量遷移至 Cloud Storage。

---

# 目錄

1. [1. 總覽](#)
2. [2. 背景](#)
3. [3. 設定/準備工作](#)
4. [4. 更新設定檔](#)
5. [5. 修改應用程式檔案](#)
6. [6. 摘要/清除](#)
7. [7. 其他資源](#)

## 1. 總覽

[無伺服器遷移工作站系列程式碼研究室](#) (自學式實作教學課程) 和[相關影片](#)，旨在協助 [Google Cloud 無伺服器](#)開發人員完成一或多項遷移作業 (主要是從舊版服務遷移)，進而翻新應用程式。這樣做可提高應用程式的可攜性，並提供更多選項和彈性，讓您整合及存取更多 Cloud 產品，並更輕鬆地升級至新版語言。雖然一開始的重點是早期 Cloud 使用者，主要是 [App Engine](#) (標準環境) 開發人員，但本系列涵蓋範圍廣泛，也包括其他無伺服器平台，例如 [Cloud Functions](#) 和 [Cloud Run](#)，或適用於其他平台。

本程式碼研究室會說明如何從 [App Engine Blobstore](#) 遷移至 [Cloud Storage](#)。此外，系統也會從下列項目進行隱含遷移：

- [webapp2 網頁架構](#)，[Flask](#) (涵蓋於[第 1 堂課](#))
- [App Engine NDB](#) 遷移至 [Cloud NDB](#)，以便存取 Datastore (涵蓋於[第 2 堂課](#))
- Python 2 至 3 (遷移後的應用程式同時與 Python 2 和 3 相容)

如需更多逐步說明，請參閱任何相關的遷移模組。

### 未使用 Blobstore 嗎？

如果您的應用程式未使用 App Engine Blobstore，可以略過本程式碼研究室，或將其視為練習，熟悉如何遷移至 Cloud Storage。

### 使用 App Engine Cloud Storage 自訂用戶端程式庫嗎？

如果您的應用程式使用 [Cloud Storage 適用的 App Engine 自訂用戶端程式庫](#) ([GoogleAppEngineCloudStorageClient](#) 或 [cloudstorage](#))，建議您也遷移至 [Cloud Storage 用戶端程式庫](#)。這類遷移作業不在本教學課程的討論範圍內，因此不會說明。[Cloud Storage 遷移頁面](#)提供了一些一般步驟。

## 在接下來的研究室中

- 新增 App Engine Blobstore API/程式庫的使用方式
- 將使用者上傳內容儲存至 Blobstore 服務
- 準備遷移至 Cloud Storage 的下一個步驟

## 軟硬體需求

- 擁有 [Google Cloud Platform 專案](#)，且已啟用 [GCP 帳單帳戶](#)
- Python 基礎技能
- 熟悉常見的 Linux 指令
- 具備[開發](#)及[部署](#) App Engine 應用程式的基本知識
- 可正常運作的第 15 模組 App Engine 應用程式：完成[第 15 模組程式碼研究室](#) (建議)，或從存放區複製[第 15 模組應用程式](#)

## 問卷調查

---

## 2. 背景

本程式碼研究室會從第 15 模組的範例應用程式開始，並示範如何從 Blobstore (和 NDB) 遷移至 Cloud Storage (和 Cloud NDB)。遷移程序包括取代 App Engine 舊版套裝服務的依附元件，讓您視需要將應用程式遷移至其他 Cloud 無伺服器平台或其他代管平台。

相較於本系列的其他遷移作業，這項遷移作業需要多花一點力氣。Blobstore 依附於原始的 webapp 架構，因此範例應用程式使用 webapp2 架構，而非 Flask。本教學課程會說明如何遷移至 Cloud Storage、Cloud NDB、Flask 和 Python 3。

應用程式仍會記錄使用者「造訪」次數，並顯示最近十次的記錄，但先前的程式碼研究室 (第 15 個) 新增了 Blobstore 用途的功能：應用程式會提示使用者上傳與「造訪」次數相應的構件 (檔案)。使用者可以選擇這麼做，也可以選取「略過」停用這項功能。無論使用者做出什麼決定，下一個頁面都會顯示與先前版本相同的輸出內容，也就是最近的造訪記錄。此外，如果造訪記錄有對應的構件，系統會顯示「查看」連結，方便您查看該造訪記錄的構件。這個程式碼研究室會實作先前提及的遷移作業，同時保留所述功能。

---

## 3. 設定/準備工作

在進入教學課程的主要部分之前，請先設定專案、取得程式碼，然後部署基準應用程式，確保我們從可運作的程式碼開始。

### 1. 設定專案

如果您已部署第 15 堂課的應用程式，建議重複使用該專案 (和程式碼)。或者，您也可以[建立全新專案](#)，或重複使用其他現有專案。確認專案已啟用 App Engine，且具備有效的帳單帳戶。

### 2. 取得基準範例應用程式

本程式碼研究室的先決條件之一，是必須有可正常運作的第 15 模組範例應用程式。如果沒有，可以從第 15 模組的「START」資料夾 (下方連結) 取得。本程式碼研究室會逐步說明每個步驟，最後的程式碼會與「Module 16」資料夾中的程式碼類似。

- 開始：[Module 15 資料夾](#) (Python 2)
- 完成：[第 16 單元資料夾](#) (Python 2)
- [整個存放區](#) (複製或下載 [ZIP 檔案](#))

模組 15 STARTING 檔案的目錄應如下所示：

```
$ ls
README.md      app.yaml      main-gcs.py   main.py       templates
```

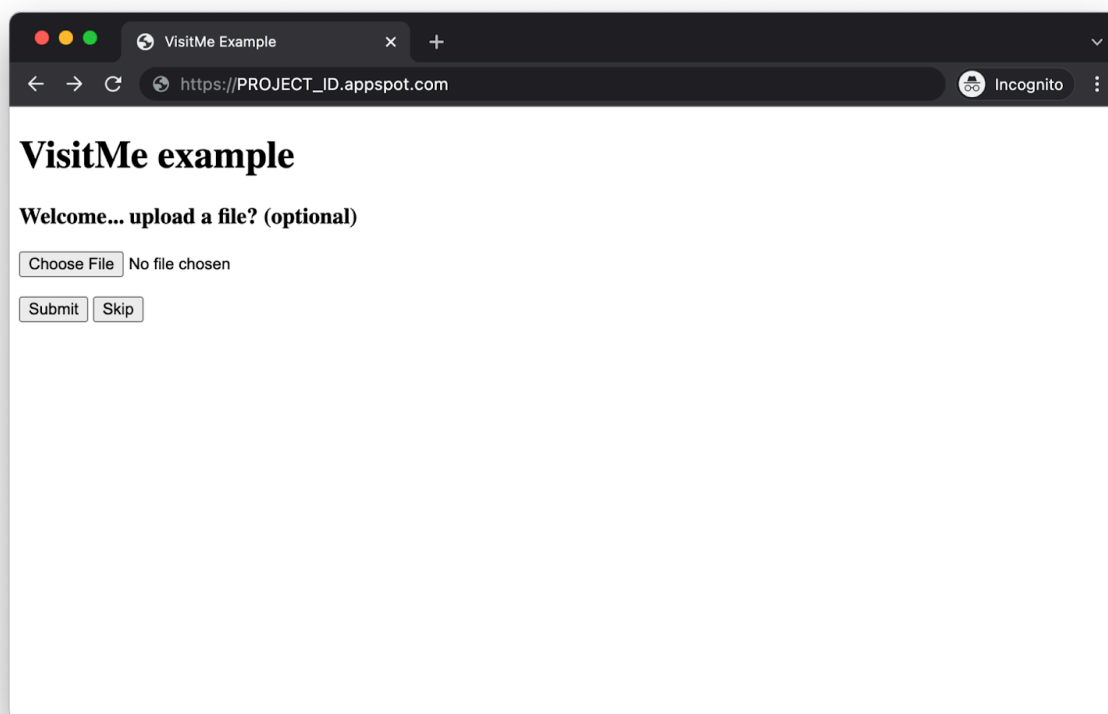
`main-gcs.py` 檔案是第 15 堂課中 `main.py` 的替代版本，可根據專案 ID ( `PROJECT_ID` .appspot.com ) 選取 Cloud Storage 值區，與應用程式指派網址的預設值不同。這個檔案與本程式碼研究室 (第 16 模組) 無關，但如果需要，類似的遷移技術也適用於該檔案。

### 3. (重新) 部署基準應用程式

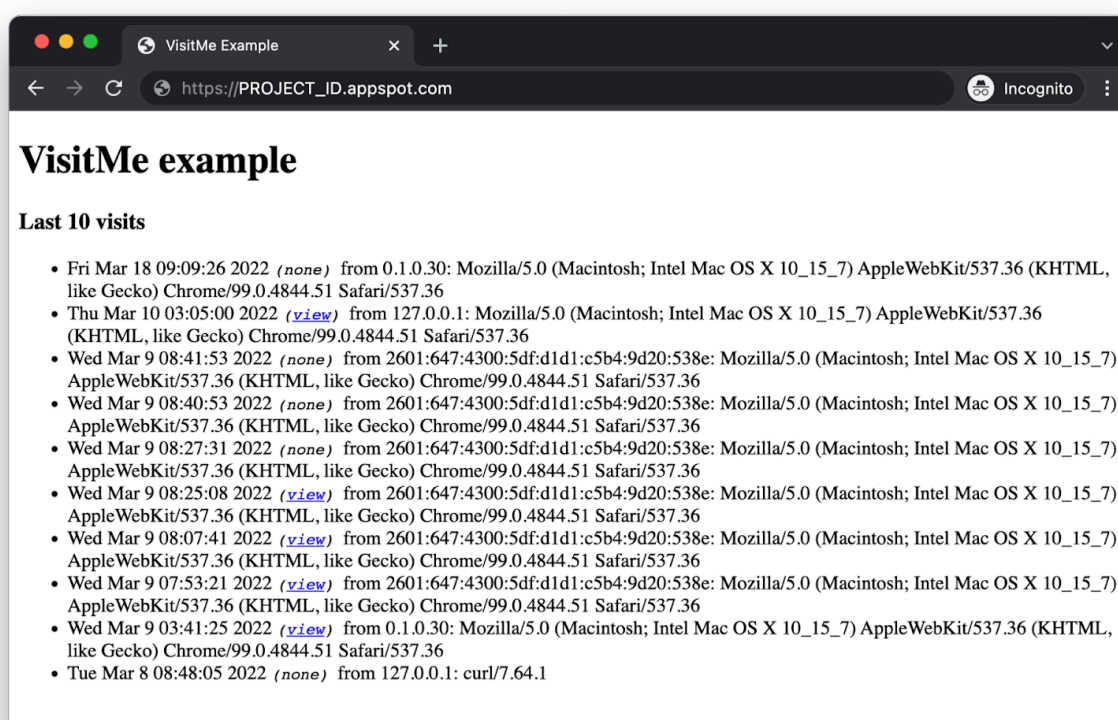
請立即執行剩餘的準備步驟：

1. 重新熟悉 `gcloud` 指令列工具
2. 使用 `gcloud app deploy` 重新部署範例應用程式
3. 確認應用程式在 App Engine 上順利執行

成功執行這些步驟並確認第 15 課的應用程式正常運作後，初始頁面會顯示表單，提示使用者上傳造訪構件檔案，並提供「略過」按鈕，讓使用者選擇不執行這項操作：



使用者上傳檔案或略過後，應用程式會顯示熟悉的「最近造訪」頁面：



如果造訪記錄包含構件，造訪時間戳記右側會顯示「查看」連結，點選即可顯示 (或下載) 構件。確認應用程式功能後，您就可以從 App Engine 舊版服務 (webapp2、NDB、Blobstore) 遷移至現代替代方案 (Flask、Cloud NDB、Cloud Storage)。

## 4. 更新設定檔

更新版應用程式會用到三個設定檔。必要工作如下：

1. 更新 `app.yaml` 中必要的內建第三方程式庫，並為 Python 3 遷移作業做好準備
2. 新增 `requirements.txt`，指定所有非內建的必要程式庫
3. 新增 `appengine_config.py`，讓應用程式同時支援內建和非內建的第三方程式庫

### app.yaml

編輯 `app.yaml` 檔案，更新 `libraries` 區段。移除 `jinja2`，並新增 `grpcio`、`setuptools` 和 `ssl`。為這三個程式庫選擇最新版本。同時新增 Python 3 `runtime` 指令，但註解掉。完成後，畫面應如下所示 (如果您選取 Python 3.9)：

#### BEFORE:

```
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app

libraries:
- name: jinja2
  version: latest
```

#### 修改後：

```
#runtime: python39
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app

libraries:
- name: grpcio
  version: latest
- name: setuptools
  version: latest
- name: ssl
  version: latest
```

這些變更主要與 [App Engine 伺服器上提供的 Python 2 內建程式庫](#) 有關 (因此您不必自行將這些程式庫與應用程式一併封裝)。我們移除了 Jinja2，因為 Flask 隨附這項功能，而我們即將將 Flask 新增至 `reqs.txt`。使用 Cloud NDB 和 Cloud Storage 等 Google Cloud 用戶端程式庫時，需要 `grpcio` 和 `setuptools`。最後，Cloud Storage 本身需要 `ssl` 程式庫。頂端註解掉的執行階段指令，是在您準備將這個應用程式移植到 Python 3 時使用。本教學課程最後會說明如何執行這項作業。

### 需要更新第三方程式庫版本嗎？

在上述 `app.yaml` 中，您指定了所要求套件的「最新」版本，也就是 Python 2 App Engine 伺服器上提供的最新版本。如果需要比[可用版本](#)更新的版本，請不要按照上述指示將其新增至 `app.yaml`。請改為在 `requirements.txt` 中列出，並按照後續指示自行組合。內建程式庫提供便利性，不必自行組合，即可使用最新版本。

## requirements.txt

新增 `requirements.txt` 檔案，其中需要 Flask 架構，以及 Cloud NDB 和 Cloud Storage 用戶端程式庫，這些都不是內建項目。建立含有以下內容的檔案：

```
flask
google-cloud-ndb
google-cloud-storage
```

Python 2 App Engine 執行階段需要自行組合非內建的第三方程式庫，因此請執行下列指令，將這些程式庫安裝到 `lib` 資料夾：

```
pip install -t lib -r requirements.txt
```

如果開發電腦同時安裝 Python 2 和 3，您可能必須使用 `pip2` 指令，確保取得這些程式庫的 Python 2 版本。升級至 Python 3 後，您就不再需要自行組合。

## appengine\_config.py

新增支援內建和非內建第三方程式庫的 `appengine_config.py` 檔案。建立含有以下內容的檔案：

```
import pkg_resources
from google.appengine.ext import vendor

# Set PATH to your libraries folder.
PATH = 'lib'
# Add libraries installed in the PATH folder.
vendor.add(PATH)
# Add libraries to pkg_resources working set to find the distribution.
pkg_resources.working_set.add_entry(PATH)
```

剛才完成的步驟應與 App Engine 文件「[為 Python 2 應用程式安裝程式庫](#)」一節中列出的步驟類似或相同，更具體來說，`appengine_config.py` 的內容應與該節的步驟 5 相同。

設定檔作業已完成，接下來請繼續處理應用程式。



## 5. 修改應用程式檔案

### 匯入

第一組變更適用於 `main.py`，包括替換所有要取代的項目。異動內容如下：

1. `webapp2` 已由 Flask 取代
2. 請使用 Flask 隨附的 Jinja2，而非 `webapp2_extras` 中的 Jinja2
3. App Engine Blobstore 和 NDB 已由 Cloud NDB 和 Cloud Storage 取代
4. `webapp` 中的 Blobstore 處理常式會由 `io` 標準程式庫模組、Flask 和 `werkzeug` 公用程式的組合取代
5. 根據預設，Blobstore 會寫入以應用程式網址 ( `PROJECT_ID.appspot.com` ) 命名的 Cloud Storage bucket。由於我們要移植到 Cloud Storage 用戶端程式庫，因此會使用 `google.auth` 取得專案 ID，以指定完全相同的 bucket 名稱。(由於儲存區名稱不再是硬式編碼，因此您可以變更儲存區名稱)。

#### BEFORE:

```
import webapp2
from webapp2_extras import jinja2
from google.appengine.ext import blobstore, ndb
from google.appengine.ext.webapp import blobstore_handlers
```

如要實作上述清單中的變更，請將 `main.py` 中的現有匯入部分，替換為下列程式碼片段。

修改後：

```
import io

from flask import (Flask, abort, redirect, render_template,
                  request, send_file, url_for)
from werkzeug.utils import secure_filename

import google.auth
from google.cloud import exceptions, ndb, storage
```

### 初始化和不必要的 Jinja2 支援

下一個要替換的程式碼區塊是 `BaseHandler`，用於指定使用 `webapp2_extras` 中的 Jinja2。Jinja2 隨附於 Flask，且是預設範本引擎，因此不需要，請移除。

在第 16 堂課中，我們將例項化舊版應用程式中沒有的物件，包括初始化 Flask 應用程式，以及為 Cloud NDB 和 Cloud Storage 建立 API 用戶端。最後，我們將 Cloud Storage 值區名稱放在一起，如上述匯入部分所述。以下是實作這些更新前後的畫面：

#### BEFORE:

```
class BaseHandler(webapp2.RequestHandler):
    'Derived request handler mixing-in Jinja2 support'
    @webapp2.cached_property
    def jinja2(self):
        return jinja2.get_jinja2(app=self.app)

    def render_response(self, _template, **context):
        self.response.write(self.jinja2.render_template(_template, **context))
```

修改後：

```
app = Flask(__name__)
ds_client = ndb.Client()
gcs_client = storage.Client()
_, PROJECT_ID = google.auth.default()
BUCKET = '%s.appspot.com' % PROJECT_ID
```

## 更新資料儲存庫存取權

Cloud NDB 大致上與 App Engine NDB 相容。我們已說明其中一項差異，也就是需要 API 用戶端。另一個差異是，後者需要由 API 用戶端的 Python 環境管理員控管 Datastore 存取權。也就是說，使用 Cloud NDB 用戶端程式庫的所有 Datastore 存取呼叫，都只能在 Python **with** 區塊內發生。

這是其中一項變更；另一項變更則是 Cloud Storage 不支援 Blobstore 及其物件 (例如 [BlobKey](#))，因此請將 `file_blob` 變更為 `ndb.StringProperty`。以下是資料模型類別，以及反映這些變更的更新版 `store_visit()` 和 `fetch_visits()` 函式：

**BEFORE:**

```
class Visit(ndb.Model):
    'Visit entity registers visitor IP address & timestamp'
    visitor = ndb.StringProperty()
    timestamp = ndb.DateTimeProperty(auto_now_add=True)
    file_blob = ndb.BlobKeyProperty()

    def store_visit(remote_addr, user_agent, upload_key):
        'create new Visit entity in Datastore'
        Visit(visitor='{}: {}'.format(remote_addr, user_agent),
              file_blob=upload_key).put()

    def fetch_visits(limit):
        'get most recent visits'
        return Visit.query().order(-Visit.timestamp).fetch(limit)
```

修改後：

```

class Visit(ndb.Model):
    'Visit entity registers visitor IP address & timestamp'
    visitor = ndb.StringProperty()
    timestamp = ndb.DateTimeProperty(auto_now_add=True)
    file_blob = ndb.StringProperty()

def store_visit(remote_addr, user_agent, upload_key):
    'create new Visit entity in Datastore'
    with ds_client.context():
        Visit(visitor='{ }: {}'.format(remote_addr, user_agent),
              file_blob=upload_key).put()

def fetch_visits(limit):
    'get most recent visits'
    with ds_client.context():
        return Visit.query().order(-Visit.timestamp).fetch(limit)

```

以下是目前為止的變更內容圖示：



## 更新處理常式

### 上傳處理常式

`webapp2` 中的處理常式是類別，而 `Flask` 中的處理常式是函式。`Flask` 會使用動詞來裝飾函式，而不是 `HTTP` 動詞方法。`Blobstore` 和其 `webapp` 處理常式已由 `Cloud Storage` 的功能以及 `Flask` 和其公用程式取代：

**BEFORE:**

```
class UploadHandler(blobstore_handlers.BlobstoreUploadHandler):
    'Upload blob (POST) handler'
    def post(self):
        uploads = self.get_uploads()
        blob_id = uploads[0].key() if uploads else None
        store_visit(self.request.remote_addr, self.request.user_agent, blob_id)
        self.redirect('/', code=307)
```

修改後：

```
@app.route('/upload', methods=[ 'POST' ])
def upload():
    'Upload blob (POST) handler'
    fname = None
    upload = request.files.get('file', None)
    if upload:
        fname = secure_filename(upload.filename)
        blob = gcs_client.bucket(BUCKET).blob(fname)
        blob.upload_from_file(upload, content_type=upload.content_type)
    store_visit(request.remote_addr, request.user_agent, fname)
    return redirect(url_for('root'), code=307)
```

本次更新的注意事項：

- 現在，檔案構件會以檔案名稱 ( `fname` ) (如有) 和 `None` (使用者選擇不上傳檔案) 識別，而非 `blob_id`。
- Blobstore 處理常式會從使用者端抽象化上傳程序，但 Cloud Storage 不會，因此您可以看到新加入的程式碼，這些程式碼會設定檔案的 Blob 物件和位置 (bucket)，以及執行實際上傳作業的呼叫。( `upload_from_file()` )。
- `webapp2` 會在應用程式檔案底部使用路由表，而 Flask 路由則位於每個裝飾處理常式中。
- 這兩個處理常式都會重新導向至首頁 ( `/` )，同時保留 `POST` 要求和 HTTP 307 傳回代碼，藉此完成功能。

## 下載處理常式

更新下載處理常式的模式與上傳處理常式類似，只是程式碼少得多。以 Cloud Storage 和 Flask 對應項目取代 Blobstore 和 `webapp` 功能：

**BEFORE:**

```
class ViewBlobHandler(blobstore_handlers.BlobstoreDownloadHandler):
    'view uploaded blob (GET) handler'
    def get(self, blob_key):
        self.send_blob(blob_key) if blobstore.get(blob_key) else self.error(404)
```

修改後：

```
@app.route('/view/<path:fname>')
def view(fname):
    'view uploaded blob (GET) handler'
    blob = gcs_client.bucket(BUCKET).blob(fname)
    try:
        media = blob.download_as_bytes()
    except exceptions.NotFound:
        abort(404)
    return send_file(io.BytesIO(media), mimetype=blob.content_type)
```

本次更新的注意事項：

- 同樣地，Flask 會使用路徑裝飾處理常式函式，而 `webapp` 則會在底部的路由表執行這項操作，因此請辨識後者的模式比對語法 ( `'/view/([^\/]*)?'`) 與 Flask 的語法 ( `'/view/<path:fname>'`)。
- 與上傳處理常式一樣，Blobstore 處理常式會抽象化功能，因此您需要在 Cloud Storage 端進行更多作業，也就是識別相關檔案 (blob) 並明確下載二進位檔，而不是呼叫 Blobstore 處理常式的單一 `send_blob()` 方法。
- 在這兩種情況下，如果找不到構件，系統會向使用者傳回 HTTP 404 錯誤。

## 主要處理常式

主要應用程式的最終變更會在主要處理常式中進行。`webapp2` HTTP 動詞方法會由單一函式取代，並合併其功能。將 `MainHandler` 類別替換為 `root()` 函式，並移除 `webapp2` 路由表，如下所示：

### BEFORE:

```
class MainHandler(BaseHandler):
    'main application (GET/POST) handler'
    def get(self):
        self.render_response('index.html',
                             upload_url=blobstore.create_upload_url('/upload'))

    def post(self):
        visits = fetch_visits(10)
        self.render_response('index.html', visits=visits)

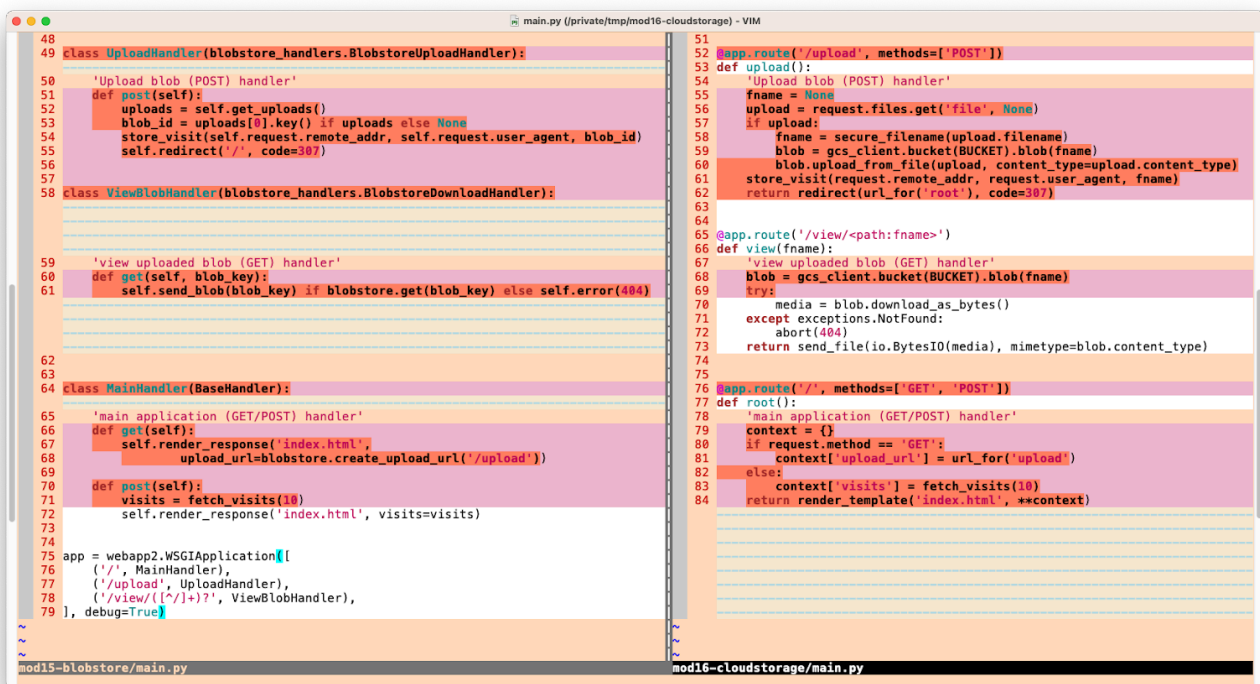
app = webapp2.WSGIApplication([
    ('/', MainHandler),
    ('/upload', UploadHandler),
    ('/view/([^\/]*)?', ViewBlobHandler),
], debug=True)
```

修改後：

```
@app.route('/', methods=['GET', 'POST'])
def root():
    'main application (GET/POST) handler'
    context = {}
    if request.method == 'GET':
        context['upload_url'] = url_for('upload')
    else:
        context['visits'] = fetch_visits(10)
    return render_template('index.html', **context)
```

這些方法並非獨立的 `get()` 和 `post()` 方法，而是 `root()` 中的 `if-else` 陳述式。此外，由於 `root()` 是單一函式，因此只需一次呼叫，即可同時為 `GET` 和 `POST` 算繪範本，但這在 `webapp2` 中並不可行。

以下是 `main.py` 第二次也是最後一次的變更內容圖示：



## (選用) 反向相容性「強化」

因此，上述解決方案非常實用，但前提是您從頭開始，且沒有 Blobstore 建立的檔案。由於我們更新了應用程式，現在會依檔案名稱而非 `BlobKey` 識別檔案，因此完成的第 16 堂課應用程式無法查看 Blobstore 檔案。換句話說，我們在執行這項遷移作業時，進行了不具回溯相容性的變更。我們現在提供 `main.py` 的替代版本，稱為 `main-migrate.py` (位於存放區中)，嘗試彌合這項差距。

支援 Blobstore 建立檔案的第一個「擴充功能」是資料模型，其中包含 `BlobKeyProperty` (以及 Cloud Storage 建立檔案的 `StringProperty`)：

```
class Visit(ndb.Model):
    'Visit entity registers visitor IP address & timestamp'
    visitor    = ndb.StringProperty()
    timestamp  = ndb.DateTimeProperty(auto_now_add=True)
    file_blob  = ndb.BlobKeyProperty() # backwards-compatibility
    file_gcs   = ndb.StringProperty()
```

`file_blob` 屬性用於識別 Blobstore 建立的檔案，`file_gcs` 則用於 Cloud Storage 檔案。現在建立新造訪時，請明確將值儲存在 `file_gcs` 中，而非 `file_blob`，因此 `store_visit` 看起來會有些不同：

#### BEFORE:

```
def store_visit(remote_addr, user_agent, upload_key):
    'create new Visit entity in Datastore'
    with ds_client.context():
        Visit(visitor='{ }: { }'.format(remote_addr, user_agent),
              file_blob=upload_key).put()
```

#### 修改後：

```
def store_visit(remote_addr, user_agent, upload_key):
    'create new Visit entity in Datastore'
    with ds_client.context():
        Visit(visitor='{ }: { }'.format(remote_addr, user_agent),
              file_gcs=upload_key).put()
```

擷取最近的造訪次數時，請先「正規化」資料，再傳送至範本：

#### BEFORE:

```
@app.route('/', methods=['GET', 'POST'])
def root():
    'main application (GET/POST) handler'
    context = {}
    if request.method == 'GET':
        context['upload_url'] = url_for('upload')
    else:
        context['visits'] = fetch_visits(10)
    return render_template('index.html', **context)
```

#### 修改後：

```
@app.route('/', methods=['GET', 'POST'])
def root():
    'main application (GET/POST) handler'
    context = {}
    if request.method == 'GET':
        context['upload_url'] = url_for('upload')
    else:
        context['visits'] = etl_visits(fetch_visits(10))
    return render_template('index.html', **context)
```

接著，確認是否存在 `file_blob` 或 `file_gcs` (或兩者皆無)。如有可用檔案，請選取現有檔案並使用該 ID (Blobstore 建立的檔案為 `BlobKey`，Cloud Storage 建立的檔案則為檔案名稱)。所謂「Cloud Storage 建立的檔案」，是指使用 Cloud Storage 用戶端程式庫建立的檔案。Blobstore 也會寫入 Cloud Storage，但在此情況下，這些是 Blobstore 建立的檔案。

現在更重要的是，這個 `etl_visits()` 函式有什麼用途？它會用於正規化或 ETL (擷取、轉換及載入) 最終使用者資料嗎？這是訂閱按鈕的圖示：

```
def etl_visits(visits):
    return [{
        'visitor': v.visitor,
        'timestamp': v.timestamp,
        'file_blob': v.file_gcs if hasattr(v, 'file_gcs') \
            and v.file_gcs else v.file_blob
    } for v in visits]
```

這可能與您預期的結果類似：程式碼會逐一檢查所有造訪記錄，並針對每筆記錄逐字擷取訪客和時間戳記資料，然後檢查是否存在 `file_gcs` 或 `file_blob`，如果存在，則選取其中一個 (如果兩者都不存在，則選取 `None`)。

以下是 `main.py` 和 `main-migrate.py` 的差異示意圖：



```
1 29 lines: Copyright 2022 Google LLC-----
30
31
32 class Visit(ndb.Model):
33     'Visit entity registers visitor IP address & timestamp'
34     visitor = ndb.StringProperty()
35     timestamp = ndb.DateTimeProperty(auto_now_add=True)
36     file_blob = ndb.StringProperty()
37
38
39 def store_visit(remote_addr, user_agent, upload_key):
40     'create new Visit entity in Datastore'
41     with ds_client.context():
42         Visit(visitor='{}: {}'.format(remote_addr, user_agent),
43             file_blob=upload_key).put()
44
45
46 def fetch_visits(limit):
47     'get most recent visits'
48     with ds_client.context():
49         return Visit.query().order(-Visit.timestamp).fetch(limit)
50
51 +-- 20 lines: @app.route('/upload', methods=['POST'])-----
71 except exceptions.NotFound:
72     abort(404)
73     return send_file(io.BytesIO(media), mimetype=blob.content_type)
74
75
76 @app.route('/', methods=['GET', 'POST'])
77 def root():
78     'main application (GET/POST) handler'
79     context = {}
80     if request.method == 'GET':
81         context['upload_url'] = url_for('upload')
82     else:
83         context['visits'] = fetch_visits(10)
84     return render_template('index.html', **context)
~
~
main.py [R0]
:1

1 29 lines: Copyright 2022 Google LLC-----
30
31
32 class Visit(ndb.Model):
33     'Visit entity registers visitor IP address & timestamp'
34     visitor = ndb.StringProperty()
35     timestamp = ndb.DateTimeProperty(auto_now_add=True)
36     file_blob = ndb.BlobKeyProperty() # backwards-compatibility
37     file_gcs = ndb.StringProperty()
38
39
40 def store_visit(remote_addr, user_agent, upload_key):
41     'create new Visit entity in Datastore'
42     with ds_client.context():
43         Visit(visitor='{}: {}'.format(remote_addr, user_agent),
44             file_gcs=upload_key).put()
45
46
47 def fetch_visits(limit):
48     'get most recent visits'
49     with ds_client.context():
50         return Visit.query().order(-Visit.timestamp).fetch(limit)
51
52 +-- 20 lines: @app.route('/upload', methods=['POST'])-----
72 except exceptions.NotFound:
73     abort(404)
74     return send_file(io.BytesIO(media), mimetype=blob.content_type)
75
76
77 def etl_visits(visits):
78     return [{
79         'visitor': v.visitor,
80         'timestamp': v.timestamp,
81         'file_blob': v.file_gcs if hasattr(v, 'file_gcs') \
82             and v.file_gcs else v.file_blob
83     } for v in visits]
84
85
86 @app.route('/', methods=['GET', 'POST'])
87 def root():
88     'main application (GET/POST) handler'
89     context = {}
90     if request.method == 'GET':
91         context['upload_url'] = url_for('upload')
92     else:
93         context['visits'] = etl_visits(fetch_visits(10))
94     return render_template('index.html', **context)
~
~
main-migrate.py [+]
```

如果您從頭開始，沒有 Blobstore 建立的檔案，請使用 `main.py`。但如果您要遷移，並希望支援 Blobstore 和 Cloud Storage 建立的檔案，請參閱 `main-migrate.py`，瞭解如何處理這類情況，協助您規劃自家應用程式的遷移作業。進行複雜的遷移作業時，很可能會發生特殊情況，因此這個範例旨在展現現代化實際應用程式和實際資料的更高親和力。

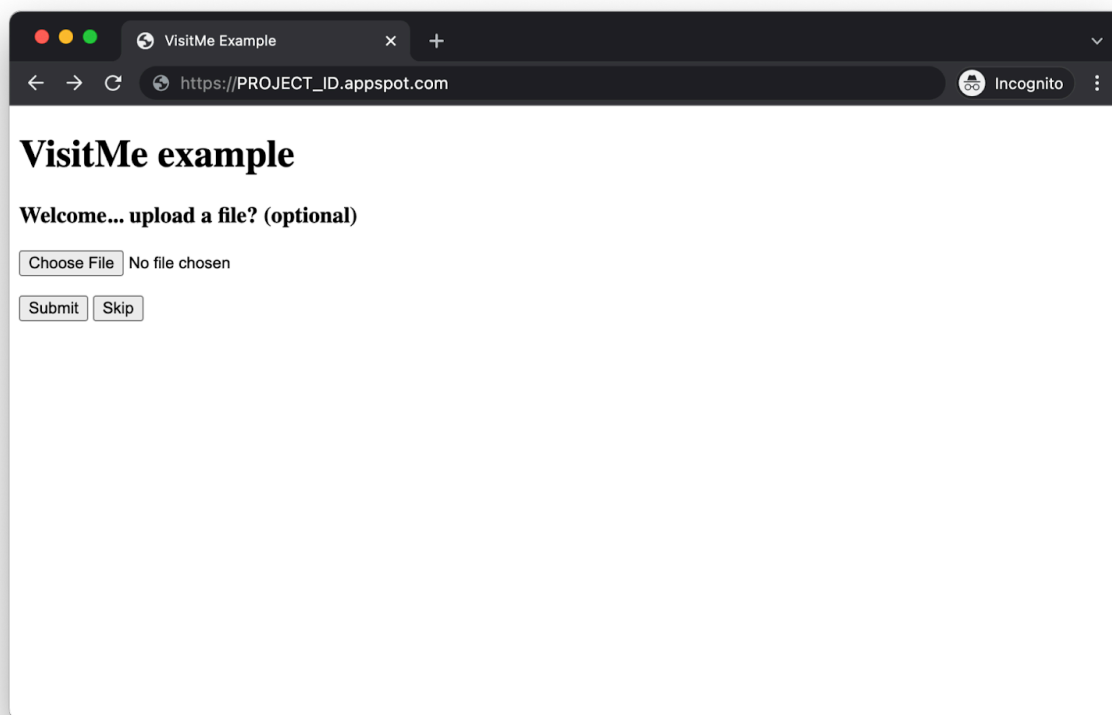
步驟 6 · 約 5 分鐘

## 6. 摘要/清除

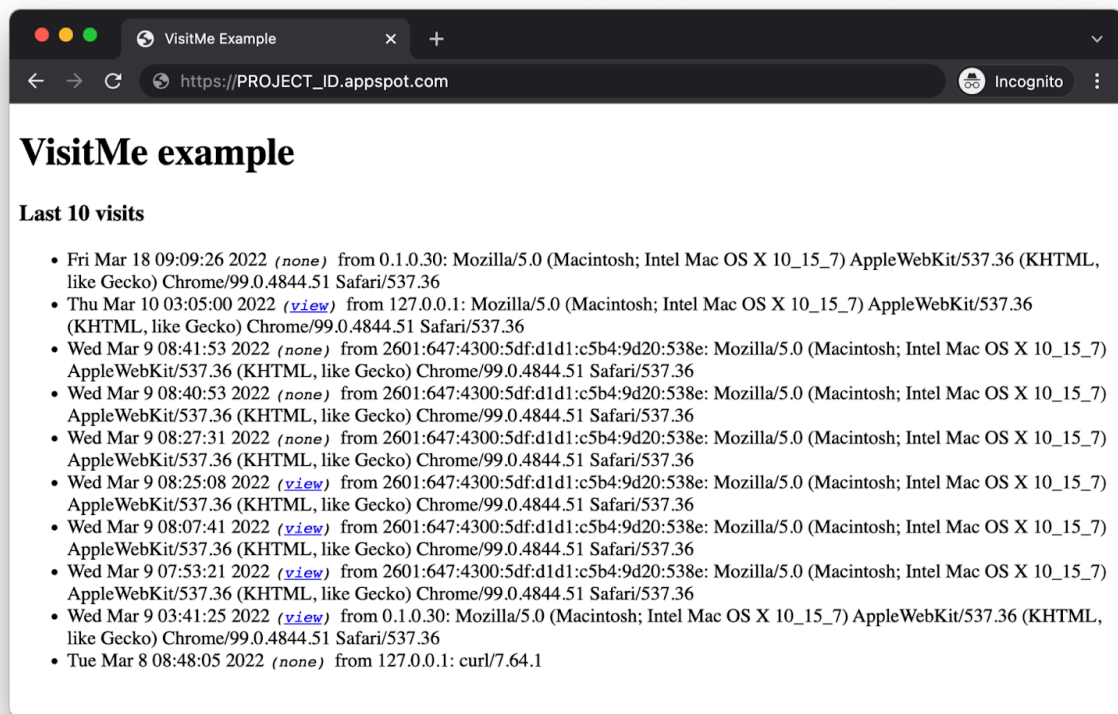
本節將部署應用程式，並驗證應用程式是否正常運作，以及是否反映在輸出內容中，為本程式碼研究室畫下句點。應用程式驗證完成後，請執行任何清理步驟，並考慮後續步驟。

### 部署及驗證應用程式

重新部署應用程式前，請務必執行 `pip install -t lib -r requirements.txt`，將這些自行組合的第三方程式庫放入 `lib` 資料夾。如要執行回溯相容解決方案，請先將 `main-migrate.py` 重新命名為 `main.py`。現在請執行 `gcloud app deploy`，並確認應用程式的運作方式與第 15 堂課的應用程式相同。表單畫面如下所示：



最近造訪的頁面如下所示：



恭喜您完成本程式碼研究室，將 App Engine Blobstore 換成 Cloud Storage、App Engine NDB 換成 Cloud NDB，並將 `webapp2` 換成 `Flask`。現在程式碼應與 [FINISH \(Module 16\) 資料夾](#) 中的程式碼相符。該資料夾中也有替代的 `main-migrate.py`。

## Python 3「遷移」

如要將這個應用程式移植到 Python 3，只需要取消註解 `app.yaml` 頂端的 Python 3 `runtime` 指令即可。原始碼本身已與 Python 3 相容，因此不需要進行任何變更。如要將這個應用程式部署為 Python 3 應用程式，請執行下列步驟：

1. 取消註解 `app.yaml` 頂端的 Python 3 `runtime` 指令。
2. 刪除 `app.yaml` 中的所有其他行。
3. 刪除 `appengine_config.py` 檔案。(Python 3 執行階段未使用)
4. 刪除 `lib` 資料夾 (如果有的話)。(使用 Python 3 執行階段時不需要)

## 清除所用資源

### 一般

如果暫時不需要使用，建議[停用 App Engine 應用程式](#)，以免產生費用。不過，如果您想進一步測試或實驗，App Engine 平台提供[免付費配額](#)，只要不超出該用量層級，就不會產生費用。這是指運算費用，但您可能也需要支付相關 App Engine 服務的費用，因此請參閱[其定價頁面](#)瞭解詳情。如果這項遷移作業涉及其他雲端服務，則這些服務會另外計費。無論是哪種情況，請視需要參閱下方的「本程式碼研究室專用」一節。

為求完整揭露，部署至 App Engine 等 Google Cloud 無伺服器運算平台時，會產生[少量建構和儲存空間費用](#)。[Cloud Build](#) 和 [Cloud Storage](#) 都有各自的免費配額。儲存該圖片會耗用部分配額。不過，你所在的區域可能沒有這類免付費層級，因此請留意儲存空間用量，盡量減少潛在費用。您應審查的特定 Cloud Storage「資料夾」包括：

- `console.cloud.google.com/storage/browser/LOC.artifacts.PROJECT_ID.appspot.com/containers/images`
- `console.cloud.google.com/storage/browser/staging.PROJECT_ID.appspot.com`

- 上述儲存空間連結取決於您的 `PROJECT_ID` 和 `*LOCATION`，例如，如果您的應用程式託管於美國，則為「us」。

另一方面，如果您不打算繼續使用這個應用程式或其他相關的遷移 Codelab，並想完全刪除所有內容，請[關閉專案](#)。

## 本程式碼研究室專用

下列服務是本程式碼研究室的專屬服務。詳情請參閱各項產品的說明文件：

- App Engine Blobstore 服務適用於[儲存資料配額和限制](#)，因此請一併參閱[舊版套裝服務的價格頁面](#)。
- Cloud Storage 在特定區域提供[免費方案](#)，詳情請參閱[一般定價頁面](#)。
- [App Engine Datastore](#) 服務是由 [Cloud Datastore](#) (Datastore 模式的 Cloud Firestore) 提供，這項服務也有免費方案；詳情請參閱[定價頁面](#)。

請注意，如果您是從第 15 集遷移至第 16 集，Blobstore 中仍會有資料，因此我們才會在上方列出 Blobstore 的價格資訊。

## 後續步驟

除了本教學課程外，您還可以參考其他遷移單元，瞭解如何從舊版套裝組合服務遷移：

- [單元 2](#)：從 App Engine `ndb` 遷移至 Cloud NDB
- [單元 7 至 9](#)：從 App Engine 工作佇列推送工作遷移至 Cloud Tasks
- [第 12 至 13 個單元](#)：從 App Engine Memcache 遷移至 Cloud Memorystore
- [單元 18-19](#)：從 App Engine 工作佇列 (提取工作) 遷移至 Cloud Pub/Sub

App Engine 不再是 Google Cloud 中唯一的無伺服器平台。如果您有小型 App Engine 應用程式或功能有限的應用程式，並希望將其轉換為獨立的微服務，或是想將單體應用程式拆分成多個可重複使用的元件，這些都是考慮遷移至 [Cloud Functions](#) 的好理由。如果容器化已成為應用程式開發工作流程的一部分，特別是如果工作流程包含 CI/CD (持續整合/持續推送軟體更新或持續部署) 管道，建議遷移至 [Cloud Run](#)。下列單元會說明這些情況：

- 從 App Engine 遷移至 Cloud Functions：請參閱[第 11 堂課](#)
- 從 App Engine 遷移至 Cloud Run：請參閱[第 4 節](#)，瞭解如何使用 Docker 將應用程式容器化；或參閱[第 5 節](#)，瞭解如何在不使用容器、Docker 知識或 `Dockerfile`s 的情況下完成這項作業。

您可以選擇改用其他無伺服器平台，但建議先考量應用程式和用途的最佳選項，再進行任何變更。

無論您接下來要考慮哪個遷移模組，都可以在 [開放原始碼存放區](#) 存取所有 [Serverless Migration Station 內容](#) (程式碼研究室、影片、原始碼 [如有])。此外，該存放區的 `README` 也提供指引，說明要考慮哪些遷移作業，以及遷移模組的相關「順序」。

---

## 7. 其他資源

### 程式碼研究室問題/意見回饋

如果發現本程式碼研究室有任何問題，請先搜尋問題，再提出回報。搜尋及建立新問題的連結：

- [搜尋問題](#)
- [回報新問題/提供意見](#)

### 遷移資源

下表提供第 15 堂 (START) 和第 16 堂 (FINISH) (FINISH) 課程的存放區資料夾連結。您也可以從[所有 App Engine Codelab](#) [遷移作業的存放區](#)存取這些範例，並複製或下載 ZIP 檔案。

| Codelab         | Python 2             | Python 3        |
|-----------------|----------------------|-----------------|
| 單元 15           | <a href="#">code</a> | 不適用             |
| 單元 16 (本程式碼研究室) | <a href="#">code</a> | (與 Python 2 相同) |

### 線上資源

以下是可能與本教學課程相關的線上資源：

#### App Engine Blobstore 和 Cloud Storage

- [App Engine Blobstore 服務](#)
- [遷移至 Cloud Storage 用戶端程式庫](#)
- [Cloud Storage 首頁](#)
- [Cloud Storage 說明文件](#)

#### App Engine 平台

- [App Engine 說明文件](#)
- [Python 2 App Engine \(標準環境\) 執行階段](#)
- [在 Python 2 App Engine 上使用 App Engine 內建程式庫](#)
- [Python 3 App Engine \(標準環境\) 執行階段](#)
- [Python 2 和 3 App Engine \(標準環境\) 執行階段的差異](#)
- [Python 2 到 3 App Engine \(標準環境\) 遷移指南](#)
- [App Engine 定價和配額資訊](#)
- [推出第二代 App Engine 平台 \(2018 年\)](#)
- [比較第一代和第二代平台](#)
- [對舊版執行階段的長期支援](#)
- [說明文件遷移範例存放區](#)
- [社群貢獻的遷移範例存放區](#)

## 其他雲端資訊

- [在 Google Cloud Platform 上執行 Python](#)
- [Google Cloud Python 用戶端程式庫](#)
- [Google Cloud 「永久免費」 方案](#)
- [Google Cloud SDK](#) ( `gcloud` 指令列工具)
- [所有 Google Cloud 說明文件](#)

## Python

- [Django](#) 和 [Jinja2](#) 範本系統
- [webapp2](#) 網頁架構
- [webapp2](#) 說明文件
- [webapp2\\_extras](#) 連結
- [webapp2\\_extras](#) Jinja2 說明文件
- [Flask](#) 網頁架構

## 影片

- [無伺服器遷移工作站](#)
- [無伺服器 Expeditions](#)
- 訂閱 [Google Cloud Tech](#)
- 訂閱 [Google Developers](#)

## 授權

這項內容採用的授權為 [Creative Commons 姓名標示 2.0 通用授權](#)。

---